CLAUDE CERTIFIED ARCHITECT — FOUNDATIONS

Objective 16: Configuration Mechanism Choice

> **Exam objective:** Select the correct Claude Code configuration mechanism—CLAUDE.md, .claude/rules/ with glob patterns, Skills, hooks, or settings permissions—based on guidance type and when it should apply.

Q1.

Your team wants every coding session in a repository to start with a short reminder about architectural boundaries, naming conventions, and the required test command. The guidance is general and should apply to all files in the repo. Which mechanism is the best fit?

A. A Skill, because it can be invoked when the model needs context
B. A repository CLAUDE.md file, because it provides broad standing guidance
C. A hook, because hooks run automatically around tool calls
D. A settings permission rule, because it controls repo behavior

Answer: B
Explanation: CLAUDE.md is the right place for durable, repository-wide guidance that should shape behavior across sessions. Skills are task-specific, hooks are for automation around execution events, and settings permissions govern tool access rather than coding guidance.

Q2.

In a monorepo, your frontend directory requires React accessibility checks, but those instructions should not apply to backend or infrastructure files. Which option is most appropriate?

A. Put all instructions in the root CLAUDE.md
B. Create a Skill named "frontend-a11y" and expect it to run automatically
C. Use .claude/rules/ with a glob pattern targeting frontend files
D. Add a settings permission override for JSX files

Answer: C
Explanation: Rules with glob patterns are designed for scoped guidance that should apply only to matching files or paths. A root CLAUDE.md would over-apply the guidance, Skills do not automatically attach based on file patterns, and permissions are unrelated to accessibility review instructions.

Q3.

An organization has a complex release checklist that should only be loaded when an engineer explicitly requests a release-preparation workflow. The checklist is too detailed to include in

always-on repository instructions. What should you use?

A. CLAUDE.md
B. A Skill
C. A hook
D. Settings permissions

Answer: B
Explanation: Skills are ideal for reusable, on-demand workflows that should not consume context in every session. CLAUDE.md is better for persistent baseline guidance, hooks automate execution events, and permissions do not encode procedural task knowledge.

Q4.

You need to automatically run a formatter after file edits and block completion if formatting fails. Which mechanism best matches this requirement?

A. A hook
B. A Skill
C. CLAUDE.md
D. .claude/rules/ with a glob pattern

Answer: A
Explanation: Hooks are intended for automatic behavior tied to events such as tool usage or task transitions. CLAUDE.md and rules provide guidance, not enforcement automation, while a Skill would require manual invocation.

Q5.

Your security team wants to prevent unrestricted shell access unless the user has explicitly allowed it in settings. This is about what tools can run, not coding conventions. Which mechanism applies?

A. Settings permissions
B. CLAUDE.md
C. A Skill
D. .claude/rules/

Answer: A
Explanation: Settings permissions control access to tools and approval behavior. CLAUDE.md, rules, and Skills influence guidance and workflows, but they do not define the permission model for shell execution.

Q6.

A data platform repo contains SQL migrations and ETL scripts. Files under migrations\ should always receive instructions about backward-compatible schema changes, but ETL scripts should not. What is the best approach?

A. Put the migration guidance in CLAUDE.md so nothing is missed
B. Use .claude/rules/ with a glob matching migrations files

C. Store the guidance in a Skill used for every task
D. Enforce it through shell permissions

Answer: B
Explanation: Scoped rules let you target the migration directory precisely without cluttering unrelated work. Putting it in CLAUDE.md would affect ETL work too, and Skills or permissions do not provide the right automatic path-based application.

Q7.

You want the model to remember that this repository prefers small pull requests, descriptive commit messages, and tests before completion. The instructions are lightweight and universally applicable. Which mechanism is best?

A. Root CLAUDE.md
B. Per-file glob rules
C. Hooks
D. Settings permissions

Answer: A
Explanation: These are general operating expectations for the repository, making CLAUDE.md the appropriate home. Glob rules are better for scoped file-specific guidance, hooks automate actions, and permissions cover access control.

Q8.

A team maintains a one-off migration assistant that generates a risk table, rollout plan, and rollback plan for legacy service cutovers. Engineers use it only a few times per quarter. Where should this logic live?

A. In CLAUDE.md so it is always available
B. In .claude/rules/ on all YAML files
C. In a Skill for explicit invocation
D. In settings permissions for deployment commands

Answer: C
Explanation: A Skill fits specialized workflows that are valuable but infrequent and too verbose for always-on context. CLAUDE.md and rules would burden normal tasks, while permissions do not capture workflow structure.

Q9.

Which scenario is the clearest sign that a hook—not guidance text—should be used?

A. "All contributors should write tests for bug fixes."
B. "Python files in api\ must use the service-layer pattern."
C. "When a task completes, run unit tests and capture the result automatically."
D. "Developers can request the incident-postmortem checklist on demand."

Answer: C
Explanation: Automatic execution on task completion is operational behavior, which is exactly what hooks are for. The other options describe general guidance, file-scoped guidance, or on-demand workflow content.

Q10.

Your repo has both Terraform and Kubernetes manifests. Terraform files need cost-awareness guidance, while Kubernetes manifests need cluster-policy reminders. What is the most maintainable choice?

A. Separate .claude/rules/ entries with appropriate glob patterns
B. One large CLAUDE.md covering every infrastructure type equally
C. A single Skill that users must remember to invoke
D. A permission setting for IaC files

Answer: A
Explanation: Separate glob-scoped rules keep guidance targeted and maintainable as file types diverge. A giant CLAUDE.md would dilute relevance, a manual Skill adds operational risk, and permissions are unrelated to domain-specific authoring advice.

Q11.

An engineering lead says, "We need something that teaches the model our house style, but it must not trigger commands or change approvals." Which option best matches that requirement?

A. CLAUDE.md or scoped rules, depending on breadth
B. Hooks
C. Settings permissions
D. Only Skills

Answer: A
Explanation: House style is guidance, so either repo-wide CLAUDE.md or path-specific rules are appropriate depending on scope. Hooks and permissions alter execution behavior, not just instruction context, and Skills are not the only answer because the requirement sounds always-on rather than optional.

Q12.

You are deciding between CLAUDE.md and a Skill for a long internal API reference that only matters when integrating with one legacy service. What is the best reasoning?

A. Prefer CLAUDE.md because more context is always better
B. Prefer a Skill because the material is specialized and should be loaded only when needed
C. Prefer a hook because API references are procedural
D. Prefer permissions because they keep the reference secure

Answer: B
Explanation: Large specialized guidance should generally be kept out of the default context unless

it is broadly relevant. A Skill preserves context efficiency, while CLAUDE.md would add noise and the other options do not fit the purpose.

Q13.

An agent may need to read sensitive local directories outside the project, but only if the user has approved such access. Which mechanism determines that?

A. A .claude/rules/ file
B. CLAUDE.md
C. Settings permissions
D. A Skill named "sensitive-reads"

Answer: C
Explanation: Whether a tool can access sensitive locations is a permissioning concern governed by settings. Guidance files and Skills can describe policy, but they do not grant or restrict the actual access level.

Q14.

Your backend team wants instructions to apply only when editing files matching ***Controller.cs, reminding the model to keep controllers thin and push logic into services. What is the best configuration?

A. Add the instruction to CLAUDE.md
B. Put it in a hook that runs before every edit
C. Create a .claude/rules/ entry scoped with an appropriate glob
D. Put it in settings permissions for C# files

Answer: C
Explanation: This is exactly the kind of targeted file-pattern guidance that scoped rules were made for. CLAUDE.md would over-apply it, hooks are the wrong abstraction, and permissions do not express architectural advice.

Q15.

Which statement best captures the decision framework for choosing among these mechanisms?

A. Use hooks for anything important because they are strongest
B. Use CLAUDE.md for broad standing guidance, rules for scoped guidance, Skills for optional workflows, hooks for automation, and settings permissions for access control
C. Use Skills instead of rules whenever glob patterns are possible
D. Use settings permissions to encode architecture decisions so they cannot be bypassed

Answer: B
Explanation: The best choice depends on whether the need is broad vs scoped, always-on vs on-demand, advisory vs automated, and instructional vs permission-related. The other options confuse distinct responsibilities and would lead to brittle or noisy configurations.